CardioIA | Parte 1 - Edge e resiliencia offline

CardioIA - Fase 3: Monitoramento Contínuo com IoT

Relatório técnico de implementação

Escopo: processamento Edge, sincronização MQTT e visualização Node-RED

Natureza dos dados: exclusivamente simulados; protótipo acadêmico sem finalidade clínica

1. Objetivo e arquitetura

A fase implementa um protótipo vestível simulado para acompanhar sinais vitais e movimento. O fluxo é composto por sensores conectados a um ESP32, lógica local de aquisição e alerta, uma fila temporária de leituras, transporte MQTT e um painel Node-RED. A separação da lógica em `edge_core.h` permite testar fila, estimativa de pulso, avaliação de limites e serialização em computador sem depender das bibliotecas físicas do microcontrolador.

O DHT22 fornece temperatura e umidade. Um botão representa cada batimento, permitindo demonstrar a estimativa de frequência cardíaca sem sensor biomédico. O MPU6050 mede aceleração e permite classificar repouso, atividade e um pico de impacto. Assim, o projeto excede o mínimo de dois sensores distintos: DHT22 e MPU6050, além da entrada digital do botão. Nenhum desses sinais simulados equivale a uma medição clínica validada.

A aplicação captura uma amostra consolidada a cada cinco segundos. O MPU6050 é consultado a cada 100 ms para aumentar a chance de observar picos de movimento entre amostras. Aquisição, leitura serial, conectividade, sincronização e atualização de indicadores são tarefas temporizadas dentro do `loop()`, sem `delay()` explícito. As chamadas de conexão da biblioteca MQTT são síncronas e podem atrasar temporariamente o loop em caso de falha de rede; o protótipo não é um sistema de tempo real certificado.

2. Parte 1: sensores e processamento local

O DHT22 é ligado ao GPIO 15 e lido em intervalo compatível com o sensor. Temperatura é armazenada em décimos de grau e umidade em porcentagem inteira para reduzir o tamanho de cada registro. Se a leitura falhar ou retornar valor inválido, os bits de estado indicam falha e a serialização MQTT representa temperatura e umidade como `null`; uma leitura defeituosa não é tratada como uma medida válida.

O botão de pulso está no GPIO 4 com `INPUT_PULLUP`. O pressionamento é ativo em nível baixo e passa por debounce temporal de 20 ms, reduzindo múltiplas contagens causadas por oscilações mecânicas. A cada pressão aceita, o estimador registra o intervalo desde o batimento anterior. A frequência é calculada a partir da média móvel de até cinco intervalos. Intervalos inferiores a 250 ms são rejeitados como repiques ou valores acima da faixa demonstrativa; pausas superiores a três

segundos reiniciam a janela. A ausência de batimento por mais de dez segundos é sinalizada como `sem_pulso`. Para facilitar testes no simulador, o comando Serial `pulso N` gera batimentos automáticos na frequência aproximada N, e `pulso 0` desliga essa função.

O MPU6050 usa I2C nos pinos SDA 21 e SCL 22 e é consultado a cada 100 ms. O firmware calcula o módulo da aceleração em unidades de gravidade e conserva o maior desvio em relação a 1 g até a próxima amostra de cinco segundos. O valor é usado para classificar atividade e gerar um alerta demonstrativo de impacto quando alcança 2,5 g. A detecção não é um algoritmo validado de queda: movimentos cotidianos podem gerar falsos positivos e a orientação do dispositivo altera a leitura.

Alertas são avaliados localmente e não dependem de internet. Os limites padrão são temperatura acima de 38 °C ou abaixo de 35 °C, frequência acima de 120 bpm ou abaixo de 50 bpm, ausência de pulso e umidade fora de 30% a 70%. Uma aceleração de impacto também gera alerta. A saída local usa LED de alerta no GPIO 25 e buzzer no GPIO 32; alertas críticos acionam o LED e o buzzer, enquanto atenção não crítica pisca. Os LEDs de conectividade e pulso usam os GPIOs 26 e 33. Os limites são parâmetros didáticos, não recomendações médicas.

3. Resiliência offline e capacidade de armazenamento

A chave deslizante no GPIO 27 e os comandos `online`/`offline` permitem simular indisponibilidade de rede. Mesmo offline, o ESP32 continua amostrando sensores, atualizando alertas locais e armazenando registros. O estado e o volume da fila podem ser consultados com `status`; o comando `limpar` descarta explicitamente a fila. A chave física volta a ser a fonte de conectividade após o comando `auto` ou ao movimentar a própria chave.

A indisponibilidade é simulada pelo booleano `wifiConectado`, conforme o enunciado. Em `offline`, a sessão MQTT é encerrada e nenhuma leitura é transmitida, mas o enlace Wi-Fi do simulador permanece associado. Essa separação evita derrubar o gateway público do Wokwi e permite demonstrar a retomada sem reiniciar a simulação. Em `online`, a sessão MQTT é reaberta e a fila é drenada em ordem.

A estratégia de capacidade é uma fila circular de 4.320 amostras. Com uma amostra a cada cinco segundos, ela cobre seis horas (`4.320 × 5 s = 21.600 s`). Cada estrutura compacta ocupa 16 bytes, totalizando 69.120 bytes para o vetor principal. Esse limite representa a hipótese de negócio do protótipo: até seis horas sem rede antes de descartar dados. Quando a fila enche, a implementação remove a amostra mais antiga e mantém as mais recentes; o contador `dropped` torna a perda visível. Essa política privilegia estado recente e não promete retenção ilimitada.

No Wokwi, o protótipo usa a RAM e a fila se perde ao encerrar ou reiniciar a simulação. O enunciado reconhece essa limitação e aceita o Monitor Serial para demonstrar a resiliência. Existe suporte opcional ao SPIFFS em `sketch.ino` (`USE_SPIFFS`): registros são acrescentados em CSV, restaurados no boot e marcados com idade desconhecida. A compactação é amortizada em blocos de 256 registros, limitando o arquivo a 4.576 linhas; essa margem não amplia a autonomia da fila RAM, que continua de seis horas. Ao esvaziar a fila sincronizada, o arquivo é removido. Reinícios durante a sincronização podem reenviar registros já publicados. O par `boot`/`seq` distingue mensagens na sessão atual, mas o identificador original de boot não é persistido no CSV; deduplicação entre reinícios não é garantida. A opção física não foi validada em hardware e requer teste de particionamento, falhas de energia e desgaste de flash antes de uso real.